A case study of balanced graph partition using simulated annealing

Sergi Monserrat Mascaró

Abstract

This report presents a simulated annealing implementation to partition the provincial map of Spain into 7 connected regions of balanced population. The problem is NP-hard and is solved with a repurposed Metropolis algorithm. The results show that the energy landscape contains an abundance of local minima separated by 22-23 flips on average, which justifies exploring a quantum version of the problem. The best result is obtained with 3 flips per iteration, with a mean energy of $(4.02 \pm 0.15) \times 10^5$ inhabitants. The optimal solution is degenerate, presenting 15 configurations with the same energy $E = 244039$ inhabitants. 14 of them share 3 out of the 7 regions, which suggests that the search could be restricted to a subspace where some regions are fixed.

Contents

1 Definition of the problem **2**

1.1 Optimality 3

1.2 Graph construction 3

2 Code description **6**

2.1 Contiguity check 7

2.2 Preservation check 7

3 Analysis and results **8**

3.1 Flips per iteration 8

3.2 Energy landscape 8

3.3 Optimal configuration 12

4 Conclusions **15**

4.1 Code improvements 16

4.2 Future plans 16

Chapter 1

Definition of the problem

This document is a detailed report on the personal coding project found in my personal GitHub repository. It was motivated by the fact that the province of Madrid contains almost 1/7 of Spain's population. Therefore, there should be an optimal way to divide the country in 7 connected regions of balanced population. Although the practical use of solving this particular problem is not immediately evident, at its core, it is still an interesting exercise on an NP-hard problem [1].

Before defining the problem, I would like to clarify some terms that will be used extensively in this report and that might lead to confusion due to their semantical similarity.

- Province: each one of the fundamental units of this problem. They are represented by a node in the graph.
- Territory: synonym of province.
- Region: group of provinces. A region can contain one or more provinces. These regions are identified by integer numbers running from 1 to 7.

1.1 Optimality

The first step to lay out the problem was to find a good definition for what constitutes an optimal partition. Ideally, there would be 7 regions with equal population. This is immediately impossible if the total population is not a multiple of 7. However, the main reason why perfectly equal populations are an unrealistic expectation is that each map configuration can vary only in discrete chunk due to each province holding upwards of 80000 inhabitants.

The immediate approach to measuring optimality, and the one I have used in this project, is to compute the deviation from the mean. That is

$$E = \sum_n |R_n - \langle R \rangle|, \tag{1.1}$$

where R_n is the population of each region and $\langle R \rangle$ is the average population per region. The goal of the simulated annealing is to minimise the quantity E, which I will refer to as the *energy* from this point on. Quantum implementations of this problem could require restating the problem as a QUBO (Quadratic Unconstrained Binary Optimisation), requiring a quadratic function for the energy. That could be solved by squaring the energy.

1.2 Graph construction

The first version of the code only optimised for E. As was to be expected, the solution it produced contained many disjointed regions, a phenomenon colloquially known as *border gore*. I did not preserve any part of that original code. Instead, I decided to add another constraint: contiguity. To be specific, all provinces belonging to a given region must share at least one border with another member of said region. This was achieved by using the `Rook` method from the Python library libpysal. This immediately posed a few problems due to some insular territories. I needed a systematic way to connect the isolated nodes that remained. libpysal includes an `attach_island` command that was initially used to connect the islands. The main limitation is that it is equivalent to a 1-nearest-neighbour connection. Therefore, strong correlations

between the islands and the point of contact with the mainland were introduced. There was a risk that the correlations would frustrate the optimisation. I decided to build a customised `attach_island` function that allowed for 3-nearest-neighbours connections. It introduced a connection between an isolated node (Ceuta, ES630) and a non-coastal province (Sevilla, ES618), but I deemed it acceptable. Thus the underlying graph of the problem was established (Figure 1.1). The graph is unweighted and symmetrical. The chosen territories for the graph partition correspond to Spain's

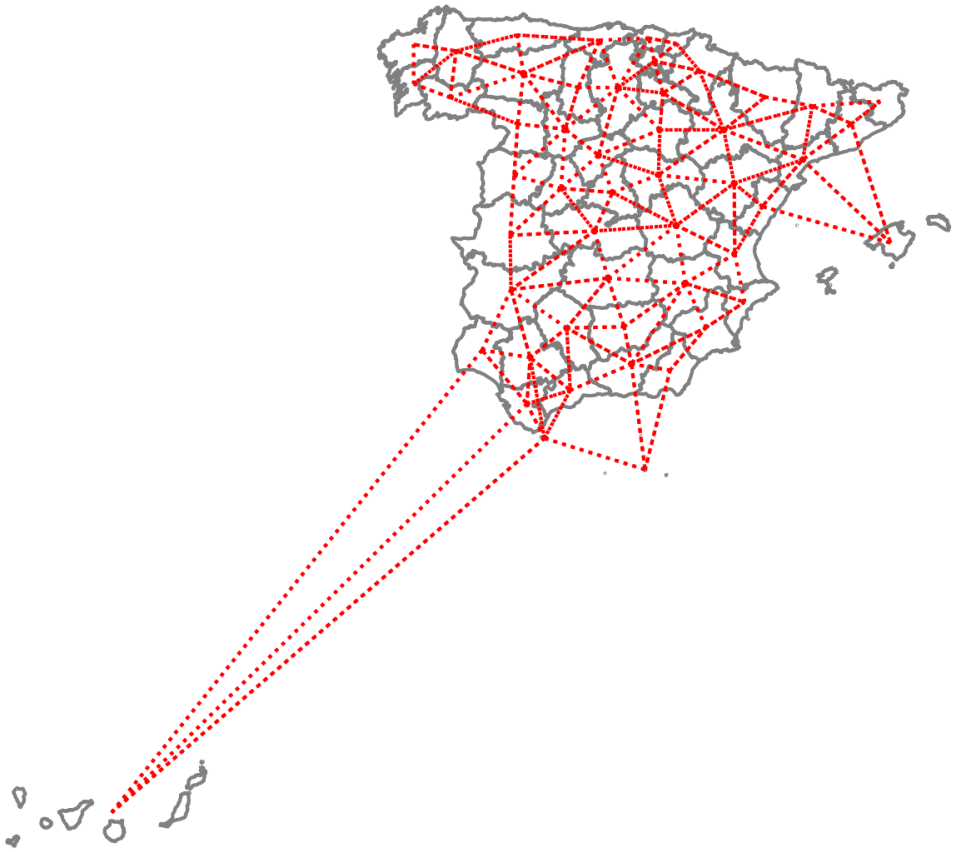


Figure 1.1: Provincial map with the graph superimposed. Note that some southern nodes correspond to territories with small land area, yet they can act as a path to connect non contiguous regions in the mainland. Sometimes solutions feature regions that may appear visually disconnected because of these difficult to see territories.

NUTS3 statistical regions but with the archipelagos fused into a single province. This is still coherent with the NUTS scheme, as the archipelagos (ES53, ES70) constitute NUTS2 regions themselves. This modification was made to avoid complications connecting so many islands both between them and to the mainland. These NUTS2 do not surpass 1/7 of the total population.

Chapter 2

Code description

The code used to solve this optimisation problem was inspired by Monte Carlo methods used to solve the 2D Ising model. Specifically, it is a repurposed Metropolis algorithm [2].

Seven seeds are planted at random on the empty map. Then, they expand until every province has been assigned to a region. For a number of iterations, a province is selected randomly and its assigned region is changed. This operation is called a *flip*.

A flip -or several of them at once- will always be accepted if it decreases the value of E. If the value of E increases after a flip there is a probability $p = \exp(-\Delta E/T)$ to accept the flip, where T is the temperature, a monotonously decreasing parameter that updates after every iteration. The cooling schedule is exponential, so a few initial steps are spent at high temperature and a majority of steps cover the lower temperatures.

In this implementation, the initial temperature is decided by sampling 1000 configurations without applying acceptance rules. The standard deviation of the samples is the initial temperature. A factor of 2 is introduced manually to promote exploration of the landscape. This often results in temperatures of the order of 10^6 inhabitants or higher. The final temperature is 1 inhabitant.

Using this setting, there is the chance to choose how many flips are made before evaluating whether to accept or reject the proposed configuration. Choosing a low

number limits exploration of the landscape and produces configurations that are correlated with one another. However, a large number of flips may promote exploration too much and kick the annealing out of promising minima, mimicking an effective temperature higher than the nominal one.

Different runs of the code can be identified by the seed used by Numpy's `random` module and the `random` library.

2.1 Contiguity check

Traditionally, these methods rely on a square grid that greatly streamlines the execution, whereas my problem is set on an irregular grid. In addition to that, my code must check every configuration for contiguity. Flips that violate this condition are never implemented in the first place and the code restarts the process of executing a flip.

The check is carried out using a depth-first search (DFS) and verifying that the number of visited nodes matches the number of provinces in that region.

2.2 Preservation check

The success of the algorithm could be compromised if a region containing only 1 province is flipped, causing the extinction of the region. That is the reason why the flip execution algorithm also includes a check to exclude single-province regions from the candidates pool.

Chapter 3

Analysis and results

3.1 Flips per iteration

I ran the annealing using 100 different seeds for 9 different values of flips/iteration, totaling 900 annealings. The seeds were always the set of integers from 0 to 99. I have found that a low number of flips is preferred, with a minimum mean energy of 402000 ± 15000 inhabitants at 3 flips/iteration. This result could be explained by the fact that 3 flips are likely to modify the composition of 6 out of 7 regions, creating enough exploration without leading to the system to behaving as if it had a much higher temperature. The difference between 2, 3, and 4 flips per iteration is not significant (Table 3.1).

3.2 Energy landscape

In order to decide whether a quantum implementation of this problem was worth considering, it was important to have a notion of what the energy landscape looks like. For this reason, I decided to compare the solutions found by my model, since each one of them could be considered a local minima. The most exhaustive approach was to find the number of flips separating every pair of solutions. This is equivalent to computing the Hamming distance between solutions. The histograms obtained for

Flips	Energy (inhabitants)	Significance
1	$(5.28 \pm 0.19) \cdot 10^5$	5.41σ
2	$(4.21 \pm 0.13) \cdot 10^5$	1.02σ
3	$(4.02 \pm 0.15) \cdot 10^5$	—
4	$(4.34 \pm 0.16) \cdot 10^5$	1.52σ
5	$(4.84 \pm 0.18) \cdot 10^5$	3.61σ
6	$(5.26 \pm 0.17) \cdot 10^5$	5.67σ
7	$(5.66 \pm 0.19) \cdot 10^5$	6.97σ
15	$(9.4 \pm 0.3) \cdot 10^5$	17.1σ
25	$(13.6 \pm 0.4) \cdot 10^5$	25.9σ

Table 3.1: Mean energy for different values of flips/iteration. Uncertainties are the mean squared error. Significance is measured with respect to the optimal case (3 flips).

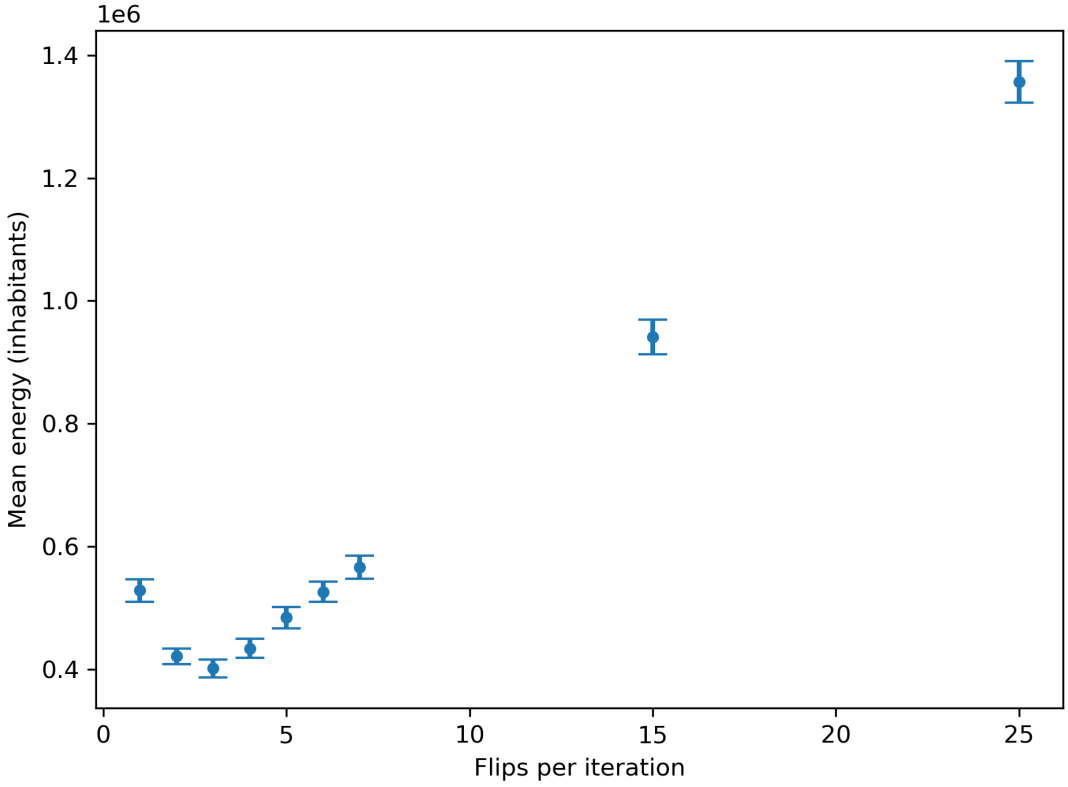
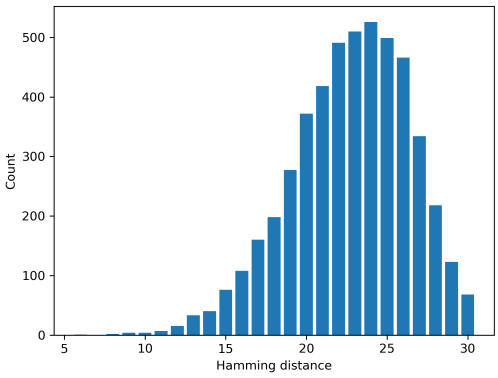
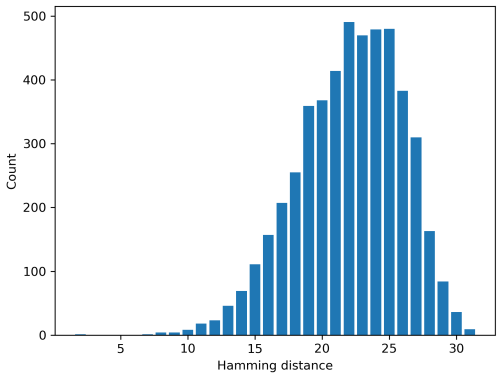


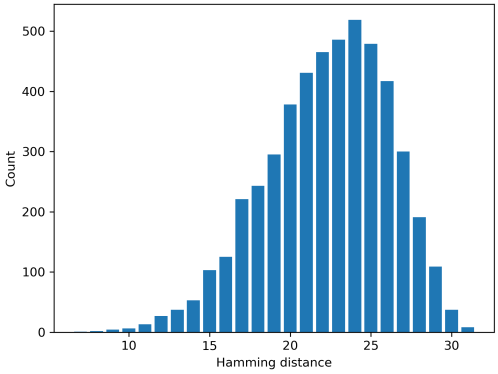
Figure 3.1: Mean energy as a function of the number of flips per iteration.



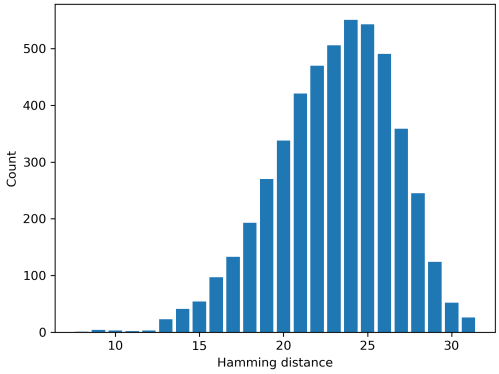
(a) 1 flip



(b) 3 flips



(c) 7 flips



(d) 25 flips

Figure 3.2: Hamming distances between pairs of solutions (seeds 0-99) for selected values of flips/iteration. Most solutions are separated by 15 to 30 flips.

Flips	Avg. Hamming distance ($\pm 1\sigma$)	Q1-Q3
1	22.72 ± 3.69	20-25
2	22.55 ± 3.78	20-25
3	22.05 ± 3.89	19-25
4	22.65 ± 3.67	20-25
5	22.50 ± 3.74	20-25
6	22.63 ± 3.71	20-25
7	22.29 ± 3.82	20-25
15	22.67 ± 3.68	20-25
25	23.01 ± 3.56	21-26

Table 3.2: Mean Hamming distance for different values of flips/iteration. The mean is presented with 1 standard deviation. Q1-Q3 represents the span of the central 50% of the distribution.

different flips/iteration values reveal a landscape where minima are separated by 15-30 flips (Figure 3.2). This separation does not depend on the number of flips/iteration and confirms that such minima are deep and difficult to escape. One could label the landscape as *tsingy-like*, imitating the sharp terrain found in the homonymous geological formation found in Madagascar, albeit in a higher dimensional space. Since there are some provinces with large populations, it was expected that some directions in this high dimensional space would resemble cliffs, but this analysis shows there are no corridors connecting the minima. If there was an easy path between minima, there would be some convergence towards the same solution and the histogram would have some population in the 0 bin. A breakdown of each histogram can be found on Table 3.2. Even though the difference in average Hamming distance may vary between values of flips/iteration, the variation itself is of the order of 1 at most. Thus, the number of flips/iteration has no effect on the average separation between solutions. It is remarkable that solutions are separated, in average, by 22 to 23 flips when the total number of nodes is 51. Thus, in average, almost half of the territories must be flipped to travel from one solution to another

It is important to stress that such large relative distances are not an artifact of the data due to differences in region labelling. The Hamming distances are computed taking into account the $7! = 5040$ possible ways to label any given map configuration. The lowest value out of all the 5040 permutations is taken as the Hamming distance.

3.3 Optimal configuration

The absolute minimum energy found by the algorithm is $E = 244039$. Out of the 900 annealings performed, there are 15 configurations that yield this value, thus the lowest energy solution is degenerate. Cases of this degeneracy have also been found at higher values of energy. Since the 15 absolute minima come from different values of flips/iteration, the same Hamming distance analysis has been performed to verify that they are all indeed different solutions (Figure 3.3). Nevertheless, a visual inspection of

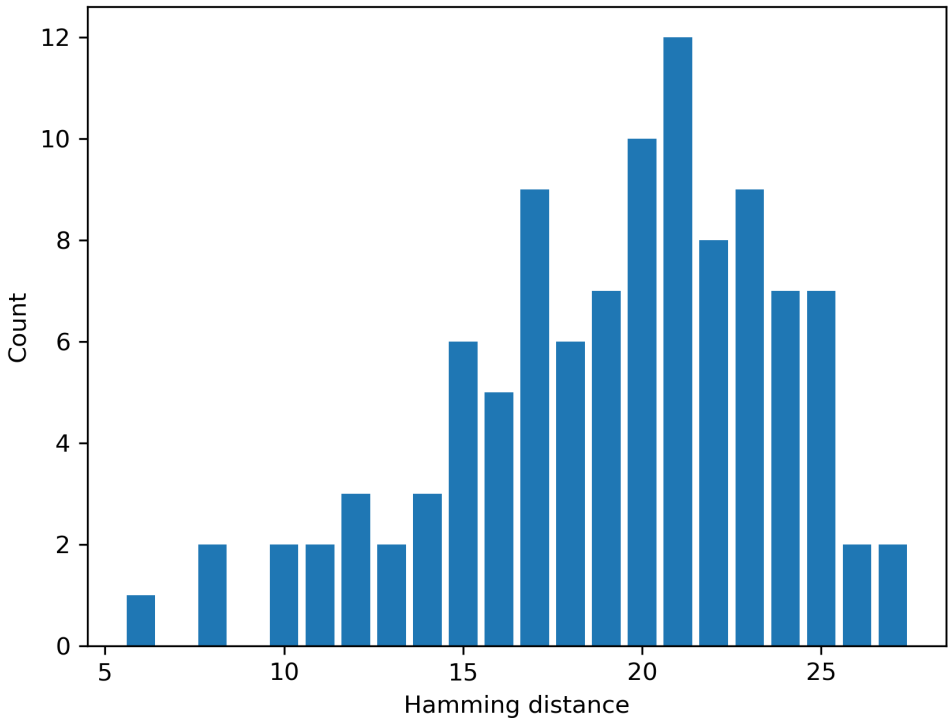


Figure 3.3: Histogram of the Hamming distances between the 15 optimal configurations.

the maps reveals that 14 out of the 15 solutions share 3 of the 7 regions (Figure 3.4). This is not surprising, given that these regions are composed by a few provinces with large populations (Madrid ES300 and Barcelona ES511) or clusters of medium-sized provinces close to each other (València ES523 + Alacant ES521 + Murcia ES620).

Therefore, the variability appears to lie in the less populated provinces. I intend to study this claim in more detail by analysing the correlations between provinces.

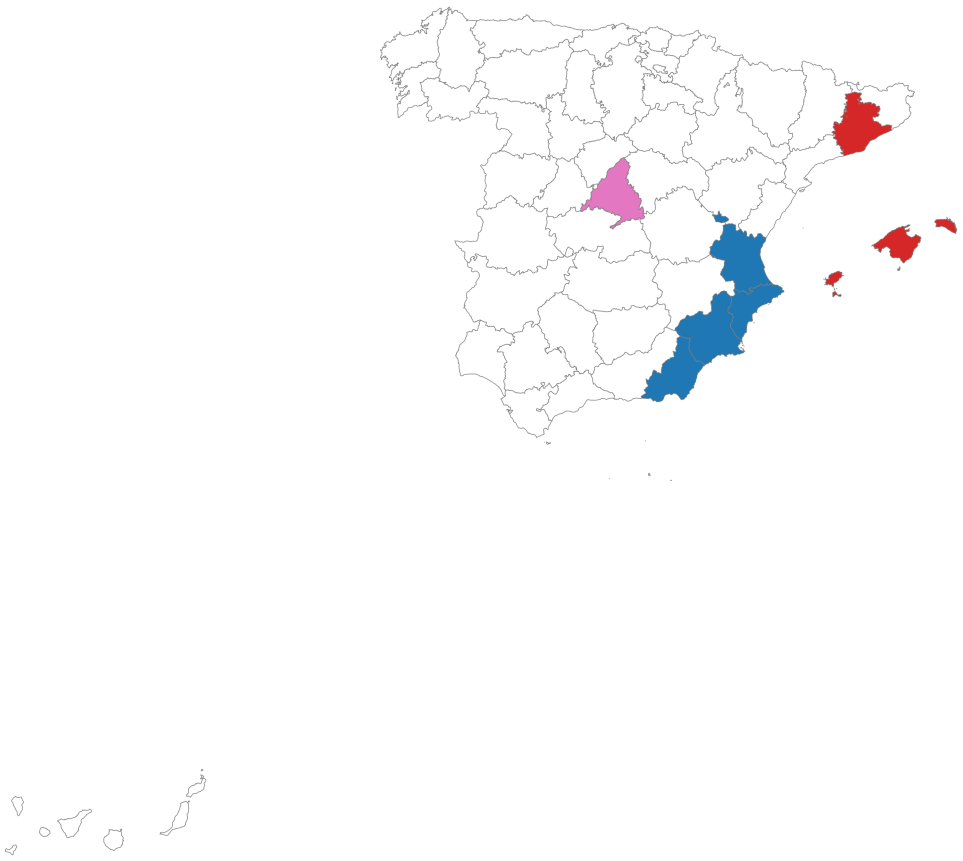
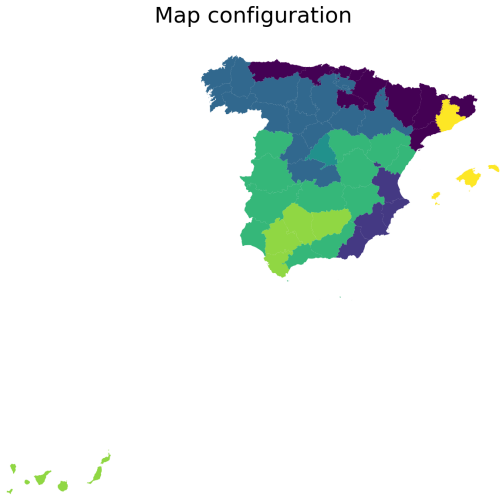
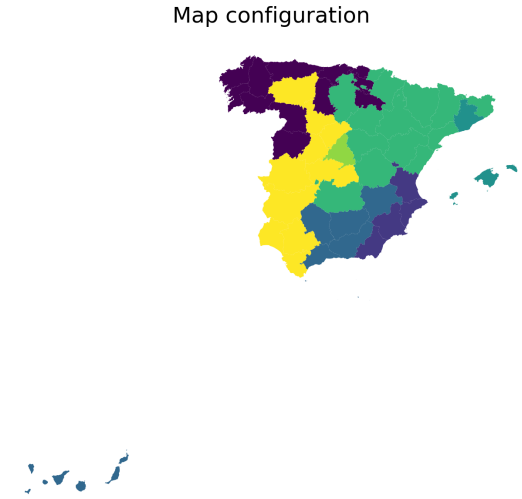


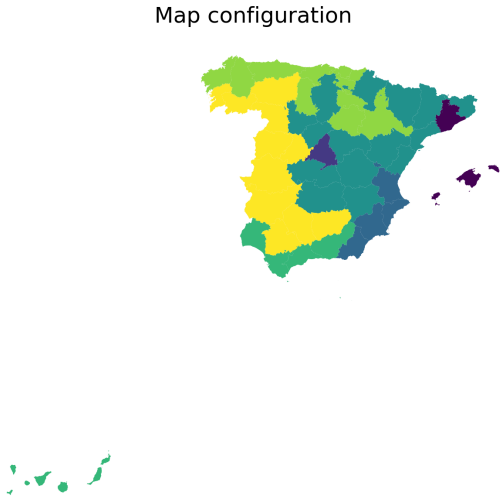
Figure 3.4: The 3 common regions in 14 out of the 15 optimal solutions. *Pink:* Madrid ES300, *red:* Barcelona ES511 and Illes Balears ES53, *blue:* València ES523, Alacant ES521, Murcia ES620, and Almería ES611



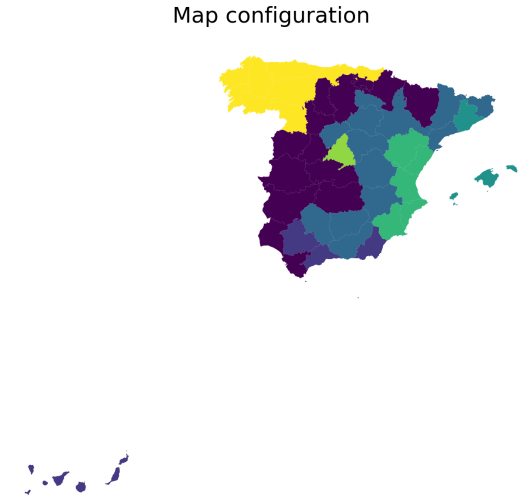
(a) Seed 26, 7 flips



(b) Seed 39, 5 flips



(c) Seed 65, 3 flips



(d) Seed 9, 4 flips

Figure 3.5: 4 examples of optimal configurations. *Bottom right:* Note that this is the only solution that does not share the 3 regions highlighted in Figure 3.4, but it still shares 2 out of the 3. At the same time, it is an example of apparent disconnection in the dark blue region due to the small area of both Ceuta ES630 and Melilla ES640.

Chapter 4

Conclusions

In this case study I have shown that partitioning the provincial map of Spain into 7 connected regions can be achieved using a simulated annealing. The associated energy landscape is complicated and difficult to explore on account of its many local minima separated by Hamming distances of 22 to 23 flips on average.

I have analysed the optimal number of flips per iteration and have found that 3 flips/iteration yields the lowest average energy of $E = (4.02 \pm 0.15) \cdot 10^5$ inhabitants across a sample of 100 seeds. However, the difference between 2, 3, and 4 flips is not statistically significant ($< 3\sigma$). Choosing a more stringent criterion ($< 5\sigma$), 5 flips is also not significantly different from 3 flips.

The most optimal solution found so far is degenerate, presenting at least 15 configurations for the same minimum value of the energy $E = 244039$ inhabitants. The solutions are proven to be distinct, but a majority of 14 out of 15 share 3 of the 7 regions. This leads to the hypothesis that the search could potentially be restricted to a subspace of all possible configurations where at least 2 of the 7 regions are fixed. Although fixing regions would speed up the annealing, further confirmation is needed before imposing restrictions that could potentially remove optimal configurations.

4.1 Code improvements

The current version of the code is poorly optimised. Despite introducing parallel computing in 12 threads, the annealing of 100 seeds for 5 values of flips/iteration still takes $\sim$ 24h 22min using an Intel Core i5-12500H CPU. I plan to introduce `numba` in as much of the code as I can. Alternatively, I might consider rewriting the entire project in C++.

I am not knowledgeable enough yet to assess if the code could benefit from GPU processing, and if so, how to implement it.

For the time being, I advise to anyone who may want to reproduce my results to abstain from inputting a large number of seeds if their computer is not prepared for it.

The auxiliary code `Hamming.py` used to compute Hamming distances also may take several hours to run. For instance, analysing 100 solutions for 4 values of flips/iteration took $\sim$ 6h on an Intel Core i5-12500H CPU.

4.2 Future plans

The scope of this project is wide and, to my pain, ever expanding. This section attempts to summarise the intended plan for the foreseeable future. Implementing every item listed may require years.

- **Characterise the graph:** For all the work I have done, I have overlooked the most basic aspect: the graph itself. I plan on studying the basic features of the graph and classifying it into one or more categories. I suspect the graph is sparse and I know it is almost planar. Given that populations often follow power laws, thoroughly characterising the graph would allow to study more general cases. In the end, I may be able to generalise this case study to sparse graphs with power law weighted nodes. In that case, I would need to test my hypotheses on artificially created graphs with the same global characteristics.
- **Study of correlations:** after finding the similarities between the 15 optimal

solutions, I thought it would be interesting to explore the commonalities between the 900 annealings I performed. The idea is to create a a correlation graph where the weights of the edges are proportional to the times two adjacent nodes ended up in the same region. I have yet to decide how to weigh each annealing so that the lowest energy solutions have a larger contribution to the graph. Once the correlation graph had been computed, the weakest links would be removed until 7 separated components formed, thus constituting a *consensus graph*.

- **Relaxing the contiguity condition:** after finding a review on quantum networks by Nokkala, Piilo and Bianconi [3], I realised I could introduce an energy function that formed the desired configuration of 7 regions at low temperatures starting from a disordered high temperature state. This, in turn, could introduce phase transitions into the model. Given the small size of the graph (~ 50 nodes), finite size effects might be prevalent.

- **Study of scalability:** how does this problem behave if the number of nodes is expanded to the entire NUTS3 regions ($\sim$ 1000 nodes)? If the number of regions is selected to be the maximum possible, then the permutations used in `Hamming.py` scale as $\mathcal{O}(n!)$. If the number of regions is kept at 7, there is more granularity and the scenario where degenerate optimal solutions share regions is less likely to happen. This point might be substituted by the study of synthetic graphs with similar characteristics as part of the generalisation discussed in the first point.

- **Implement a quantum version of the problem:** a problem with many local minima separated by difficult to traverse walls is the precise setting where quantum tunnelling might prove faster and more efficient. On top of that, a full quantum treatment could exhibit different phase transitions compared to the classical version. So far, I have considered implementing the quantum annealing taking inspiration from Hartmann et al. [4].

Bibliography

[1] M.E. Dyer and A.M. Freize. “On the complexity of partitioning graphs into connected subgraphs”. In: *Discrete Applied Mathematics* 10 (1985), pp. 139–152. URL: https://doi.org/10.1016/0166-218X(85)90008-3.

[2] Nicholas Metropolis et al. “Equation of State Calculations by Fast Computing Machines”. In: *The Journal of Chemical Physics* 21 (1953), pp. 1087–1092. URL: https://doi.org/10.1063/1.1699114.

[3] Johannes Nokkala, Jyrki Piilo and Ginestra Bianconi. “Complex Quantum Networks: A Topical Review”. In: *Journal of Physics A: Mathematical and Theoretical* 57 (2024), p. 233001. URL: https://doi.org/10.1088/1751-8121/ad41a6.

[4] Carsten Hartmann et al. “Quantum Annealing Based Power Grid Partitioning for Parallel Simulation”. In: *IEEE Transactions on Power Systems* 40 (2025), pp. 4958–4970. URL: https://doi.org/10.1109/TPWRS.2025.3578243.